



DEPARTAMENTO
DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Trabajo práctico 1

AED2

September 10, 2026

Algoritmos y Estructuras de Datos

laJavaneta

Integrante	LU	Correo electrónico
Tamara Argañaraz	1189/24	tamaradiana.ar@gmail.com
Fran	/	@
Marti	/	@
Fernández Fazio, Adrián Patricio	938/24	adrianfernandezfazio@gmail.com



Facultad de Ciencias Exactas y Naturales

Universidad de Buenos Aires

Ciudad Universitaria - (Pabellón I/Planta Baja)

Intendente Güiraldes 2610 - C1428EGA

Ciudad Autónoma de Buenos Aires - Rep. Argentina

Tel/Fax: (+54 +11) 4576-3300

<http://www.exactas.uba.ar>

enum Categoria ES {Bronce, Plata, Oro, Platino}

Vuelos ES seq($\mathbb{R}$)

Millas ES $\mathbb{R}$

Usuario ES $\mathbb{Z}$

MillasGanadas ES seq(struct <millas : Millas, nombrePromo : string, factorPromo : $\mathbb{R}$ >>

MillasCanjeadas ES seq(Millas)

MillasRecibidas ES seq(struct <transferidor : Usuario, millas : Millas>>

MillasTransferidas ES seq(struct <receptor : Usuario, millas : Millas>>

Transacciones ES struct <vuelos : Vuelos, millasGanadas : MillasGanadas, millasCanjeadas : MillasCanjeadas, millasRecibidas : MillasRecibidas, millasTransferidas : MillasTransferidas>

Usuarios ES dicc(usuario : Usuario, transacciones : Transacciones)

Características ES struct <factor : $\mathbb{R}$, tope : $\mathbb{R}$ >

Promos ES dicc(nombre : string, factor : $\mathbb{R}$)

PromosConTope ES dicc(nombre : string, características : Características)

TAD AirMiles {

obs usuarios : Usuarios

obs promos : Promos

obs promosConTope : PromosConTope

proc crearSistema() : AirMiles {

requiere { true }

asegura {

res.usuarios = {} $\wedge$

res.promos = <<nombre : 'NoPromo', factor : 1>> $\wedge$

res.promosConTope = <>

}

}

proc registrarUsuario(inout aerolínea : AirMiles, in usuario : Usuario) {

requiere {

usuario $\notin$ aerolínea.usuarios $\wedge$ aerolínea = aerolínea0

}

asegura {

aerolínea.usuarios = setKey(aerolínea0.usuarios, usuario, <vuelos : <>, millasGanadas : <>, millasCanjeadas : <>, millasRecibidas : <>, millasTransferidas : <>>) $\wedge$

aerolínea.promos = aerolínea0.promos $\wedge$

aerolínea.promosConTope = aerolínea0.promosConTope

}

}

```

proc registrarUsuarioConReferidor(inout aerolínea : AirMiles,) {
    in usuario : Usuario,
    in referidor : Usuario

    requiere {
        aerolínea = aerolínea0  $\wedge$  usuario  $\notin$  aerolínea.usuarios  $\wedge$  referidor  $\in$  aerolínea.usuarios
    }
    asegura {
        aerolínea.usuarios = setKey(agregarUsuarioReferido(aerolínea0.usuarios, usuario),
        referidor,  $\langle$ vuelos : aerolínea0.usuarios[referidor].vuelos,
        millasGanadas : concat(aerolínea0.usuarios[referidor].millasGanadas,
         $\langle$ millas : 500, nombrePromo : 'NoPromo', factorPromo : 1) $\rangle$ ,
        millasCanjeadas : aerolínea0.usuarios[referidor].millasCanjeadas,
        millasRecibidas : aerolíneas0.usuarios[referidor].millasRecibidas,
        millasTransferidas : aerolínea0.usuarios[referidor].millasTransferidas $\rangle$ )  $\wedge$ 
        aerolínea.promos = aerolínea0.promos  $\wedge$ 
        aerolínea.promosConTope = aerolínea0.promosConTope
    }
}

aux agregarUsuarioReferido(usuarios : Usuarios, usuario : Usuario) : Usuarios =
    setKey(usuarios, usuario,  $\langle$ vuelos :  $\langle$ ,
    millasGanadas :  $\langle$ millas : 500, nombrePromo : 'NoPromo', factorPromo : 1 $\rangle$ ,
    millasCanjeadas :  $\langle$  $\rangle$ , millasRecibidas :  $\langle$  $\rangle$ , millasTransferidas :  $\langle$  $\rangle$  $\rangle$ )

proc acumularMillas(inout aerolínea : AirMiles, ) {
    in usuario : Usuario,
    in distanciaVolada : Vuelos,
    in promo : string

    requiere {
        aerolínea = aerolínea0  $\wedge$  usuario  $\in$  aerolínea.usuarios  $\wedge$  (promo  $\in$  aerolínea0.promos  $\vee$ 
        promo  $\in$  aerolínea0.promosConTope)  $\wedge$ 
        distanciaVolada > 0
    }
    asegura {
        aerolínea.usuarios = agregarNuevaTransacción(aerolínea0.usuarios, usuario,
        distanciaVolada, promo, IfThenElse(promo  $\in$  aerolínea0.promos,
        aerolínea0.promos[promo], aerolínea0.promosConTope[promo].factor)  $\wedge$ 
        aerolínea.promosConTope = actualizarPromoConTope(promo,
        aerolínea.usuarios[usuario], aerolíneas.promosConTope)  $\wedge$ 
        aerolíneas.promos = aerolíneas0.promos
    }
}

```

```

aux agregarNuevaTransacción(usuarios : Usuarios, usuario : Usuario, distanciaVolada :
Vuelos, promo : string, factor : ℝ) : Usuarios =
  setKey(usuarios, usuario, ⟨vuelos : concat(aerolínea0.usuarios[usuario].vuelos,
  ⟨distanciaVolada⟩),
  millasGanadas : concat(usuarios[usuario].millasGanadas,
  ⟨agregarMillasGanadas(usuarios[usuario], distanciaVolada, promo)⟩),
  millasCanjeadas : usuarios[usuario].millasCanjeadas,
  millasRecibidas : usuarios[usuario].millasRecibidas,
  millasTransferidas : usuarios[usuario].millasTransferidas)

aux agregarMillasGanadas(distanciaVolada : Vuelos, transacciones : Transacciones,
promo : string, factor : ℝ) : MillasGanadas =
  ⟨millas : calcularMillasAcumuladas(distanciaVolada, transacciones, factor),
  nombrePromo : promo, factorPromo : factor⟩

aux calcularMillasAcumuladas(distanciaVolada : Vuelos, transacciones : Transacciones,
factor : ℝ) : ℝ =
  distanciaVolada * factorCategoría(transacciones) * factor

aux factorCategoría(transacciones : Transacciones) : ℝ =
  IfThenElse(esBronce(millasTotales(transacciones)), 1,
    IfThenElseFi(esPlata(millasTotales(transacciones)), 1.5,
      IfThenElseFi(esOro(millasTotales(transacciones)), 2, 0)))

pred esBronce(millasTotales : ℝ) {
  millasTotales < 10000
}

pred esPlata(millasTotales : ℝ) {
  10000 ≤ millasTotales < 50000
}

pred esOro(millasTotales : ℝ) {
  50000 ≤ millasTotales < 1000000
}

proc transferirMillas(inout aerolínea : airMiles, in usuarioTransferidor : Usuario,) {
  in usuarioReceptor : Usuario, in millasATransferir : Millas
  requiere {
    aerolínea = aerolínea0 ∧ usuarioTransferidor ∈ aerolínea.usuarios ∧
    usuarioReceptor ∈ aerolínea.usuarios ∧
    millasATransferir ≤ millasDisponibles(aerolínea0.usuarios[usuarioTransferidor])
  }
  asegura {

```

```

    aerolínea.usuarios = setKey(agregarTransferenciaRecibida(aerolínea0.usuarios,
    usuarioTransferidor, usuarioReceptor, millasATransferir),
    usuarioTransferidor, {vuelos : aerolínea0.usuarios[usuarioReceptor].vuelos,
    millasGanadas : aerolínea0.usuarios[usuarioTransferidor].millasGanadas,
    millasCanjeadas : aerolínea0.usuarios[usuarioTransferidor].millasCanjeadas,
    millasRecibidas : aerolíneas0.usuarios[usuarioTransferidor].millasRecibidas,
    millasTransferidas :
    concat(aerolíneas0.usuarios[usuarioTransferidor].millasTransferidas,
    {receptor : usuarioReceptor, millas : millasATransferir}})) ∧
    aerolínea.promos = aerolínea0.promos ∧
    aerolínea.promosConTope = aerolínea0.promosConTope
  }
}

```

aux agregarTransferenciaRecibida(*usuarios* : Usuarios, *usuarioTransferidor* : Usuario, *usuarioReceptor* : Usuario, *millasATransferir* : Millas) : Usuarios =

```

    setKey(usuarios, usuarioReceptor, {vuelos : usuarios[usuarioReceptor].vuelos,
    millasGanadas : usuarios[usuarioReceptor].millasGanadas,
    millasCanjeadas : usuarios[usuarioReceptor].millasCanjeadas,
    millasRecibidas : concat(usuarios[usuarioReceptor].millasRecibidas,
    {transferidor : usuarioTransferidor, millas : millasATransferir}}))
    millasTransferidas : usuarios[usuarioReceptor].millasTransferidas,

```

```

proc consultarSaldo(inout aerolínea : AirMiles, in usuario : Usuario) : Millas {
  requiere {
    usuario ∈ aerolínea.usuarios
  }
  asegura {
    res = millasDisponibles(aerolínea.usuarios[usuario])
  }
}

```

```

proc rankingMillas(in aerolinea :, ∈ cantidad : ℤ) : seq{<> } {
  requiere {
    cantidad ≥ 0
  }
  asegura {
    ((|res| = cantidad) ∨ (|res| = |aerolinea.usuario| ∧ cantidad = 0) ∧ ((usuario :
    ℤ ∧ Lusuario ∈ aerolinea.usuarios) ⇒ (∃ n : ℤ)(m : ℤ)(0 ≤ n, m < |res| ⇒
    L(ifThenElse((n > m) ∧ (millasTotales(aerolinea.usuarios[n]) > millasTotales(aerolinea.usu
    ))
  }
}

```